图灵机

概念

图灵机形式定义：

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

各符号释义：

- Γ：纸带符号集，满足 $\Sigma \subseteq \Gamma$
- $B \in \Gamma \setminus \Sigma$：空白符号
- $\delta : Q \times \Gamma \rightharpoonup Q \times \Gamma \times \{L, R\}$：转移函数（确定型图灵机），这里的 δ 是**偏函数**

若当前“状态—纸带符号”没有对应转移，机器立即停机。用于语言识别时，停机于接受状态表示接受，停机于其他状态表示拒绝；因此“停机”不等同于“接受”。

- 纸带：双向无限长

常见变形图灵机：多带图灵机、非确定图灵机、多轨图灵机、单向无限纸带图灵机

- 多道图灵机：共 k 道，单个读头单次读取 k 个符号
- 多带图灵机：拥有 k 条独立纸带与 k 个读写头

对角化方法揭示了算法固有的局限性：存在图灵机无法判定的问题。先区分两个常用语言：

- $A_{TM} = \{\langle M, w\rangle \mid M \text{ 接受} w\}$，成员条件是 M 在输入 w 上进入接受状态；
- $HALT_{TM} = \{\langle M, w\rangle \mid M \text{ 在} w \text{ 上停机}\}$，成员条件只要求停机，无论最终接受还是拒绝。

因此，A_{TM} 与 $HALT_{TM}$ 的成员条件不同，不能把“停机”与“接受”混为一谈。

1. **递归可枚举集** RE（Recursively Enumerable）

存在图灵机接受该语言：输入串属于语言时，图灵机停机并接受；输入串不属于语言时，图灵机可以拒绝，也可以永不停机。

2. **递归集（可判定集** Decidable）

存在图灵机对任意输入总能停机，并明确输出**接受/拒绝**（全域停机图灵机）。

$HALT_{TM}$ **不可判定的反证：**假设存在判定器 H 判定 $HALT_{TM}$，即 $H(\langle M, w\rangle)$ 总能停机，并正确判断 M 在 w 上是否停机。构造反转机 D：输入 $\langle M\rangle$ 后，调用 $H(\langle M, \langle M\rangle\rangle)$；若 H 判定 M 在 $\langle M\rangle$ 上停机，则 D 进入死循环；若 H 判定 M 在 $\langle M\rangle$ 上不停机，则 D 立即停机。最后考察 $D(\langle D\rangle)$：若 H 判定它停机，按 D 的构造它反而不停机；若 H 判定它不停机，按构造它反而停机。两种情况都矛盾，因此判定器 H 不存在，$HALT_{TM}$ 不可判定。

归约（多对一归约 $\leq_m$**）**

若 $A \leq_m B$（问题 A 可有效转化为问题 B）：

- 若 B 可判定 $\implies$ A 可判定；
- 逆否命题：若 A 不可判定 $\implies$ B 不可判定。

实际证明中，通常将已知不可判定问题归约到待证问题，以此证明目标问题不可判定。

构造思路

设计思路不唯一，但不同思路构造起来所需状态有差异，即使都可以实现相同功能

子串	思路
计数	以 $\{0^n1^n\}$ 为例：1. 处于 $q0$, 若待读入的为 0，则将 0 标记为 X, 向右移动 2. 一直向右移动 (跳过 0 和 Y)，直到遇到第一个 1，标记为 Y, 改为向左移动 3. 一直向左移动 (跳过 Y 和 0)，直到遇到 X, 改为向右移动 4. 如果下一个要读入的符号是 0，则跳转到第一步；否则，向右移动检查是否还有剩余的 1。如果后面已经全是 Y，那么在空格符停下，进入接受状态。
识别回文	记忆读取的字符，移动到右边进行匹配，重复操作
识别重复串	有标记：直接标记前面再到后面比对无标记：需要两头跑找到中点（或对两部分打上前后标记）
插入字符	移动搬运字符

例题：重复串识别

$$L = \{01^n0^{2m}1^n \mid n, m \geq 1\}$$

该语言要求：

- 以 0 开头；
- 后跟 n 个 1 ($n \geq 1$)；
- 后跟 2m 个 0 ($m \geq 1$，即至少两个且总数为偶数的 0)；
- 最后是 n 个 1，与左侧 1 块等长。

设计思路

采用**分阶段标记匹配法**：

1. 先验证输入以 0 开头，并验证左侧 1 块非空。
2. 每轮把左侧最左边尚未标记的 1 改写为 X，向右越过左侧剩余的 1 和中间 0 区，再越过右侧已经标记的 Y，把右侧最左边尚未标记的 1 改写为 Y。
3. 返回纸带左端，重复第 2 步。匹配时，中间 0 只被越过，绝不作为匹配目标。
4. 左侧 1 全部标记为 X 后，从中间 0 区的第一个 0 开始验证：0 的数量必须至少为 2 且为偶数。
5. 最后验证右侧只剩 Y，随后读到空白符 B 才接受；若仍有未标记的 1、出现多余 0 或提前读到 B，则拒绝。

本节采用偏转移函数：q_{accept} 与 q_{reject} 都是停机状态，没有外出转移；表中列出的拒绝转移在进入 q_{reject} 后停机。

状态集 Q

- q_0：初始状态，验证首字符为 0
- q_{first}：验证左侧 1 块非空，并标记第一个左侧 1
- $q_{seekMid}$：越过左侧剩余的 1，寻找中间 0 区
- $q_{scanMid}$：越过中间 0 区，寻找右侧 1 块
- $q_{seekRight}$：越过已标记的 Y，寻找右侧尚未标记的 1
- q_{back}：向左返回纸带左端
- q_{resume}：重新越过开头的 0
- $q_{findLeft}$：越过 X，寻找下一个尚未标记的左侧 1

- $q_{zeroOdd}$、$q_{zeroEven}$：验证中间 0 区至少有两个 0 且长度为偶数
- $q_{checkRight}$：验证右侧只含 Y
- q_{accept}：接受停机状态
- q_{reject}：拒绝停机状态

状态转移表

状态	读入	写入	移动	下一状态	注释
q_0	0	0	R	q_{first}	验证首字符为 0
q_0	1	1	R	q_{reject}	不以 0 开头
q_0	B	B	R	q_{reject}	空串
q_{first}	1	X	R	$q_{seekMid}$	左侧 1 块非空，标记第一个 1
q_{first}	0	0	R	q_{reject}	左侧 1 块为空
q_{first}	B	B	R	q_{reject}	输入不完整
$q_{seekMid}$	1	1	R	$q_{seekMid}$	越过左侧尚未标记的 1
$q_{seekMid}$	0	0	R	$q_{scanMid}$	进入中间 0 区
$q_{seekMid}$	X	X	R	q_{reject}	结构错误
$q_{seekMid}$	Y	Y	R	q_{reject}	缺少中间 0 区
$q_{seekMid}$	B	B	R	q_{reject}	缺少中间 0 区或右侧 1
$q_{scanMid}$	0	0	R	$q_{scanMid}$	越过中间 0；奇偶性稍后单独验证
$q_{scanMid}$	1	Y	L	q_{back}	标记右侧第一个尚未标记的 1
$q_{scanMid}$	Y	Y	R	$q_{seekRight}$	右侧已有标记，继续寻找未标记的 1
$q_{scanMid}$	X	X	R	q_{reject}	结构错误
$q_{scanMid}$	B	B	R	q_{reject}	右侧 1 数量不足
$q_{seekRight}$	Y	Y	R	$q_{seekRight}$	越过右侧已标记的 1
$q_{seekRight}$	1	Y	L	q_{back}	标记右侧下一个尚未标记的 1
$q_{seekRight}$	0	0	R	q_{reject}	右侧 1 块中出现 0
$q_{seekRight}$	X	X	R	q_{reject}	结构错误
$q_{seekRight}$	B	B	R	q_{reject}	右侧 1 数量不足
q_{back}	0	0	L	q_{back}	向左返回
q_{back}	1	1	L	q_{back}	向左返回
q_{back}	X	X	L	q_{back}	向左返回
q_{back}	Y	Y	L	q_{back}	向左返回
q_{back}	B	B	R	q_{resume}	到达纸带左端
q_{resume}	0	0	R	$q_{findLeft}$	越过开头的 0
q_{resume}	1	1	R	q_{reject}	首字符结构被破坏
q_{resume}	B	B	R	q_{reject}	输入不完整
$q_{findLeft}$	X	X	R	$q_{findLeft}$	越过已匹配的左侧 1
$q_{findLeft}$	1	X	R	$q_{seekMid}$	标记下一个左侧 1
$q_{findLeft}$	0	0	R	$q_{zeroOdd}$	左侧匹配完毕；已读入第 1 个中间 0
$q_{findLeft}$	Y	Y	R	q_{reject}	缺少中间 0 区
$q_{findLeft}$	B	B	R	q_{reject}	输入不完整
$q_{zeroOdd}$	0	0	R	$q_{zeroEven}$	读入第 2、4、6……个中间 0
$q_{zeroOdd}$	1	1	R	q_{reject}	中间 0 数量为奇数或右侧未完全匹配
$q_{zeroOdd}$	Y	Y	R	q_{reject}	中间 0 数量为奇数
$q_{zeroOdd}$	X	X	R	q_{reject}	结构错误
$q_{zeroOdd}$	B	B	R	q_{reject}	缺少右侧 1 块
$q_{zeroEven}$	0	0	R	$q_{zeroOdd}$	继续计数中间 0

状态	读入	写入	移动	下一状态	注释
$q_{zeroEven}$	Y	Y	R	$q_{checkRight}$	中间 0 为正偶数，检查右侧标记
$q_{zeroEven}$	1	1	R	q_{reject}	右侧还有未匹配的 1
$q_{zeroEven}$	X	X	R	q_{reject}	结构错误
$q_{zeroEven}$	B	B	R	q_{reject}	缺少右侧 1 块
$q_{checkRight}$	Y	Y	R	$q_{checkRight}$	越过已匹配的右侧 1
$q_{checkRight}$	1	1	R	q_{reject}	右侧 1 数量多于左侧
$q_{checkRight}$	0	0	R	q_{reject}	右侧 1 块后出现 0
$q_{checkRight}$	X	X	R	q_{reject}	结构错误
$q_{checkRight}$	B	B	R	q_{accept}	结构、匹配数与中间 0 奇偶性均正确

除表中列出的转移外，其余“状态—读入符号”组合均无转移并停机拒绝。

逐步走带验证

配置记号 $\alpha\, q\, \beta$ 表示纸带非空内容为 $\alpha\beta$，读头扫描 β 的首符号；B 表示空白。按语言定义，$n = m = 1$ 的合法串是 01001（$0 \cdot 1 \cdot 00 \cdot 1$），其运行如下：

q0 01001
⊢ 0 q_first 1001
⊢ 0X q_seekMid 001
⊢ 0X0 q_scanMid 01
⊢ 0X00 q_scanMid 1
⊢ 0X0 q_back 0Y
⊢ 0X q_back 00Y
⊢ 0 q_back X00Y
⊢ q_back 0X00Y
⊢ q_back B0X00Y
⊢ q_resume 0X00Y
⊢ 0 q_findLeft X00Y
⊢ 0X q_findLeft 00Y
⊢ 0X0 q_zeroOdd 0Y
⊢ 0X00 q_zeroEven Y
⊢ 0X00Y q_checkRight B
⊢ 0X00YB q_accept B

因此 01001 被接受。题目纠错条目中把 010011 写成“$01^1 0^2 1^1$”是字符计数笔误：010011 实际分解为 $0{\cdot}1{\cdot}00{\cdot}11$，左右 1 块长度分别为 1 和 2，不属于 L。本机在完成一对匹配后，$q_{checkRight}$ 会读到多余的未标记 1，并转入 q_{reject}。

另一个非法串 0101 的中间 0 区只有 1 个 0。匹配左右两个 1 后，$q_{zeroOdd}$ 直接读到 Y，因中间 0 数量为奇数而转入 q_{reject}。

UML 状态图

图中只画出主流程；所有未画出的非法输入均转入停机状态 q_{reject}。

```
stateDiagram-v2
  direction LR
  [*] --> q0
  q0 --> q_first: 0/0,R
```

q_first --> q_seekMid: 1/X,R
q_seekMid --> q_seekMid: 1/1,R
q_seekMid --> q_scanMid: 0/0,R
q_scanMid --> q_scanMid: 0/0,R
q_scanMid --> q_back: 1/Y,L
q_scanMid --> q_seekRight: Y/Y,R
q_seekRight --> q_seekRight: Y/Y,R
q_seekRight --> q_back: 1/Y,L
q_back --> q_back: 0,1,X,Y / 原符号,L
q_back --> q_resume: B/B,R
q_resume --> q_findLeft: 0/0,R
q_findLeft --> q_findLeft: X/X,R
q_findLeft --> q_seekMid: 1/X,R
q_findLeft --> q_zeroOdd: 0/0,R
q_zeroOdd --> q_zeroEven: 0/0,R
q_zeroEven --> q_zeroOdd: 0/0,R
q_zeroEven --> q_checkRight: Y/Y,R
q_checkRight --> q_checkRight: Y/Y,R
q_checkRight --> q_accept: B/B,R
q_accept --> [*]: 接受
q_reject --> [*]: 拒绝

例题：连续匹配

$L = \{w \mid w \in \{0,1\}^* \text{ 且 } w \text{ 中至少有 3 个连续的 1}\}$

该语言要求字符串中包含子串 “111” (三个连续的 1)。

设计思路

采用**状态记忆法**:

- 使用状态来记录已经连续读到的 1 的个数
- 当连续读到 3 个 1 时，立即接受
- 如果遇到 0，重置计数器

状态集 Q

q0: 初始状态 - 未读到 1 q1: 已读到 1 个连续的 1 q2: 已读到 2 个连续的 1 q3: 已读到 3 个连续的 1（接受停机状态）q_reject: 拒绝停机状态（到达磁带末尾仍未找到 111）

状态转移表

状态	读入	写入	移动	下一状态	注释
q0	0	0	R	q0	读到 0，保持状态，右移
q0	1	1	R	q1	读到第 1 个连续的 1
q0	B	B	R	q_reject	到达末尾仍未找到 111
q1	0	0	R	q0	遇到 0，重置计数
q1	1	1	R	q2	读到第 2 个连续的 1
q1	B	B	R	q_reject	到达末尾仍未找到 111

状态	读入	写入	移动	下一状态	注释
q2	0	0	R	q0	遇到 0，重置计数
q2	1	1	R	q3	读到第 3 个连续的 1，进入接受停机状态
q2	B	B	R	q_reject	到达末尾仍未找到 111

接受状态 q3 与拒绝状态 q_reject 都没有外出转移。

stateDiagram-v2
direction LR
[*] --> q0
q0: q0 无连续 1
q1: q1 1 个连续 1
q2: q2 2 个连续 1
q3: q3 已找到 3 个连续 1【接受】
qrej: qrej 拒绝

q0 --> q0 : 0/0,R
q0 --> q1 : 1/1,R
q0 --> qrej : B/B,R
q1 --> q0 : 0/0,R
q1 --> q2 : 1/1,R
q1 --> qrej : B/B,R
q2 --> q0 : 0/0,R
q2 --> q3 : 1/1,R
q2 --> qrej : B/B,R
q3 --> [*] : 接受
qrej --> [*] : 拒绝

例题：偶长度回文串识别

$$L = \{ww^R \mid w \in \{0,1\}^*\}$$

该语言要求字符串是某个字符串 w 与其反转 w^R 的连接，因此只包含偶长度回文串；本例不识别奇长度回文串。

设计思路

采用**双指针匹配法**：

1. 首先检查字符串长度是否为偶数
2. 使用标记法，从左半部分取一个符号，与右半部分对应位置的符号比较
3. 如果所有对应位置都匹配，则接受

形式定义

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

- 状态集 $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_{\text{accept}}, q_{\text{reject}}\}$
- 输入字母表 $\Sigma = \{0, 1\}$

- 带字母表 $\Gamma = \{0, 1, X, Y, B\}$，$B$ 为空白符
- 初始状态 q_0
- 接受状态 q_{accept}，拒绝状态 q_{reject}

转移函数 δ

下表只讨论语言 $L = \{ww^R\}$，即偶长度回文串。采用标准 L/R 移动；q_{accept} 与 q_{reject} 为没有外出转移的停机状态。

状态	读入	写入	移动	下一状态	注释
q_0	0	X	R	q_1	记录左端符号 0
q_0	1	Y	R	q_2	记录左端符号 1
q_0	B	B	R	q_{accept}	所有字符均已成对处理
q_1	0/1/X/Y	不变	R	q_1	向右寻找末尾
q_1	B	B	L	q_3	到达末尾，准备匹配 0
q_2	0/1/X/Y	不变	R	q_2	向右寻找末尾
q_2	B	B	L	q_4	到达末尾，准备匹配 1
q_3	0	B	L	q_5	右端与记录的 0 匹配
q_3	其他	不变	R	q_{reject}	右端不匹配
q_4	1	B	L	q_5	右端与记录的 1 匹配
q_4	其他	不变	R	q_{reject}	右端不匹配
q_5	0/1	不变	L	q_5	向左返回
q_5	X/Y	不变	R	q_6	越过最左标记
q_6	0	X	R	q_1	开始下一轮，记录符号 0
q_6	1	Y	R	q_2	开始下一轮，记录符号 1
q_6	B	B	R	q_{accept}	偶数个字符均已成对处理
q_6	其他	不变	R	q_{reject}	结构错误

工作流程

1. **开始** (q_0)：读最左未标记字符，是 0 则写 X 并进入 q_1，是 1 则写 Y 并进入 q_2；若读到空白则接受（处理完所有字符）。
2. **向右找右端** (q_1/q_2)：无视所有符号右移，直到遇到空白，然后左移一格，进入检查状态 q_3（期待 0）或 q_4（期待 1）。
3. **匹配与擦除** (q_3/q_4)：若右端符号与记录的左端符号一致，则将其改写为空白，并进入 q_5 向左返回；否则拒绝。
4. **向左返回** (q_5)：连续左移越过 0、1，直到碰到 X 或 Y（最左标记），然后右移一格到下一个未处理字符，进入 q_6。
5. **判断下一轮**（q_6）：若该位置是 0 或 1，直接标记该符号并开始下一轮；若是空白，说明偶数个字符已成对处理完毕，进入接受停机状态。

例题：移动删除操作

设计一台图灵机，输入字母表为 $\Sigma = \{0, 1\}$，输入为任意二进制串。实现功能：删除输入串中所有 0，将剩余所有 1 在纸带中连续紧凑输出；若输入为空串，直接停机。（本题只要求计算完成并最终停机，无需显式定义单独接受状态。）

图灵机形式化定义

这是**计算函数/字符串变换**场景：停机表示输出已经计算完成，而不是语言识别意义上的“接受”。若把它改作语言识别器，则必须另设显式接受状态；在语言识别语境中，停机不等于接受。

设计计算型图灵机 $M = (Q, \Sigma, \Gamma, \delta, q_0, B, \emptyset)$，各参数定义如下：

- **状态集**：$Q = \{q_0, q_1, q_2, q_3, q_4\}$
- **输入字母表**：$\Sigma = \{0, 1\}$
- **纸带字母表**：$\Gamma = \{0, 1, X, Y, B\}$（$B$ 为空白符，X Y 为辅助标记符）
- **初始状态**：q_0
- **空白符号**：B
- **接受状态集**：$\emptyset$。本题不判定语言成员关系，只以正常停机表示字符串变换完成
- **转移函数**：偏函数，由状态转移表确定；没有对应规则时停机

阶段分析

该图灵机通过**标记清零、逐位搬运、符号还原**三个核心阶段，实现删除所有 0、聚合所有 1 的功能，具体过程如下：

阶段一：标记所有输入 0（q0→q1） 机器从初始状态 q_0 启动，从纸带最左端开始向右扫描输入串：读到 0 则改写为辅助标记 X，读到 1 则保留原符号，随后进入 q_1。状态 q_1 持续向右扫描整个输入串，将所有 0 统一替换为 X，所有 1 保持不变。当带头扫描至输入串末尾、读到空白符 B 时，停止右移并左移一位，进入状态 q_2。此阶段完成后，原输入中所有 0 被标记、所有 1 完整保留，区分出有效字符与待删除字符。

阶段二：逐位搬运 1 至纸带右端（q2↔q3 循环） 状态 q_2 负责向左回溯扫描，跳过所有辅助标记 X、Y，寻找纸带中剩余的有效字符 1。当找到 1 后，将该 1 改写为 X（标记该位置 1 已被搬运），随后右移进入 q_3。

状态 q_3 持续向右扫描，跳过所有 X、Y 标记，直至读到最右端的空白符 B，将空白符改写为 Y（标记搬运后的 1 的位置），之后左移回到 q_2。

机器通过 q_2 与 q_3 循环往复，将纸带中所有 1 逐一搬运至串的最右端，并用 Y 标记位置，保证搬运后的 1 相互连续。当 q_2 向左扫描至纸带最左端、读到空白符 B 时，说明所有 1 已全部搬运完成，此时右移进入清理状态 q_4。

阶段三：清理辅助标记、还原有效字符（q4） 状态 q_4 从左至右全域扫描纸带，完成符号还原与清理：将所有占位标记 X 擦除为空白符 B，将所有代表搬运 1 的标记 Y 还原为有效字符 1。当扫描至纸带末尾的空白符 B 时，无对应转移规则，机器自动停机。最终纸带仅保留一串连续的 1，其余位置均为空白，达成题目要求。

边界情况处理 **空输入情况**：若输入为空串，机器启动后在初始状态 q_0 直接读到空白符 B；该转移不存在，机器正常停机，表示空串的变换计算完成。这里不把停机称为语言识别意义上的接受。

极端输入兼容：输入全 0 时，所有字符会被标记为 X，搬运阶段无 1 可处理，最终清理后纸带全部为空白；输入全 1 时，无 0 需要删除，搬运后 1 保持连续，不改变原有效字符。

状态转移规则

转移函数 δ 完整定义如下。移动列中的“停机”表示该转移不存在；其余已定义转移只使用标准方向 L/R。

状态	读入	写入	移动	下一状态	注释
q0	0	X	R	q1	标记输入 0

状态	读入	写入	移动	下一状态	注释
q0	1	1	R	q1	保留输入 1
q0	B	B	停机	—	空输入，计算完成；转移不存在
q1	0	X	R	q1	标记输入 0
q1	1	1	R	q1	保留输入 1
q1	B	B	L	q2	到达输入末尾
q2	X	X	L	q2	向左寻找尚未搬运的 1
q2	Y	Y	L	q2	向左寻找尚未搬运的 1
q2	1	X	R	q3	标记并搬运一个 1
q2	B	B	R	q4	所有 1 均已搬运
q3	X	X	R	q3	向右寻找输出末尾
q3	Y	Y	R	q3	向右寻找输出末尾
q3	B	Y	L	q2	在末尾追加一个 1 的标记
q4	X	B	R	q4	清除占位标记
q4	Y	1	R	q4	还原输出字符 1
q4	B	B	停机	—	计算完成；转移不存在

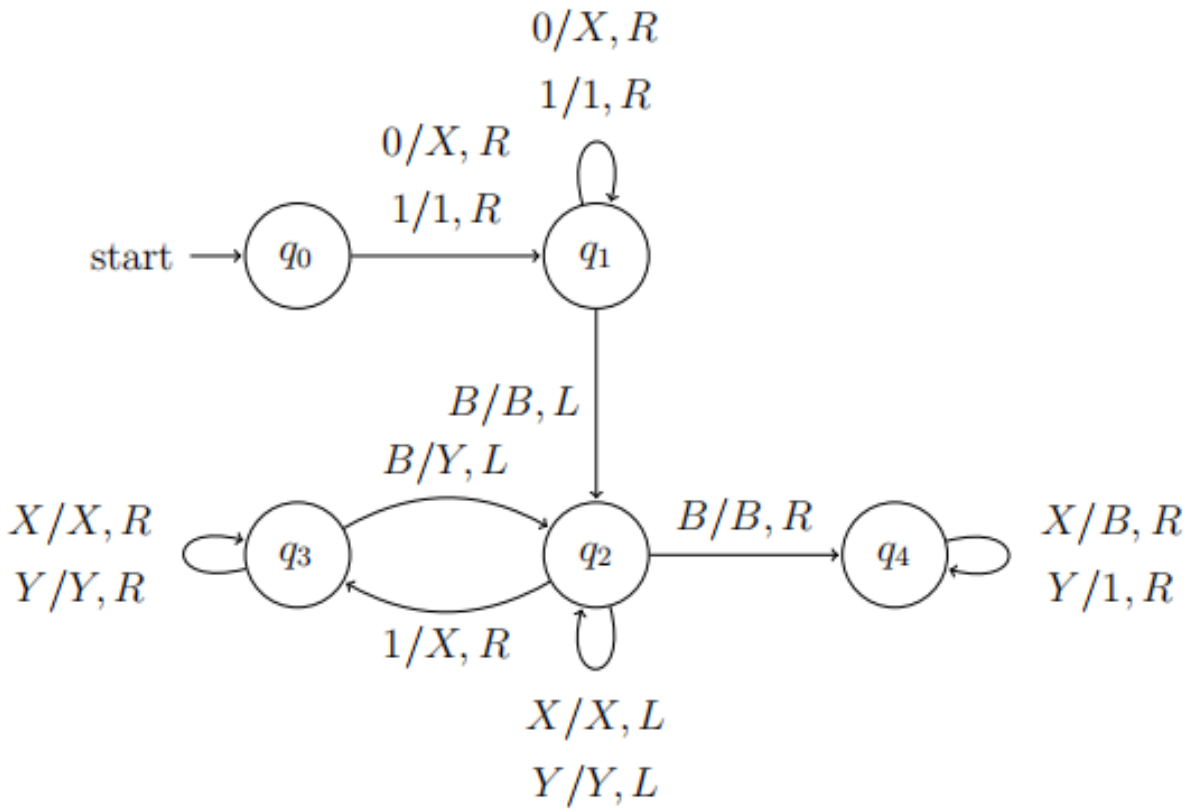


Figure 1: image.png

例题：计算加法

设计一个图灵机，将输入的二进制数加 1。例如，输入 101（即十进制 5），输出 110（即十进制 6）。假设输入只包含 0 和 1，且没有前导 0（除了 0 本身）。

1. 纸带字母表 $\Gamma = \{0, 1, \sqcup\}$

- 输入字母表 $\Sigma = \{0, 1\}$，$\sqcup$：空白

2. 状态集合 $Q = \{q_0, q_1, q_h\}$
   - q_0：初始状态（向右移动到数字最右端最低位）
   - q_1：进位处理核心状态（向左逐位翻转）
   - q_h：停机接受状态，没有外出转移
3. 初始状态：q_0；停机状态：q_h
4. 移动指令：L 左移，R 右移

转移函数 $\delta : Q \times \Gamma \rightharpoonup Q \times \Gamma \times \{L, R\}$ 为偏函数，转移表如下：

状态	读入	写入	移动	下一状态	注释
q_0	0	0	R	q_0	右移，寻找串末尾空白
q_0	1	1	R	q_0	右移，寻找串末尾空白
q_0	$\sqcup$	$\sqcup$	L	q_1	到达数字最右侧，左移开始进位
q_1	1	0	L	q_1	当前位是 1，加 1 变 0，继续向左进位
q_1	0	1	R	q_h	当前位是 0，加 1 变 1，无进位，进入停机接受状态
q_1	$\sqcup$	1	R	q_h	全部位都是 1，左侧空白写 1，进入停机接受状态

q_h 没有外出转移；进入 q_h 即停机接受。转移逻辑拆解

1. **阶段 1 q_0：右移到数字末尾**

从头向右遍历所有 $0/1$，直到读到空白，此时磁头停在二进制串最右侧数字的右边空白，左移一位进入进位状态 q_1

例：输入 101，磁头初始在第一个 1，右移依次经过 0,1，读到末尾空白，左移到最右位 1。

2. **阶段 2 q_1：进位向左遍历**
   - 读到 1：置 0，继续左移（进位传递）
   - 读到 0：置 1，直接停机（进位终止）
   - 读到空白：说明所有位都是 1，在空白处写 1，停机（溢出补位）

思考题：

1. 纸带输入两个二进制数，如 BB101+110=BB，要求在等号右边给出计算结果
2. 计算乘法
3. 计算 n!
4. 计算 2^n